

Working Flows Guide

End-to-end walkthrough of 8 modules

1.	real_estate_base	Property catalog & map
2.	atmta_real_estate	Rental contracts
3.	real_estate_brokerage	Sales & brokerage
4.	real_estate_developer	Projects & inventory
5.	real_estate_construction	Construction tracking
6.	real_estate_handover	Handover & warranty
7.	real_estate_investment	NPV / IRR / DCF
8.	real_estate_portal	Customer self-service

Version 0.1
Generated 2026-05-13
Author: Atmta

Contents

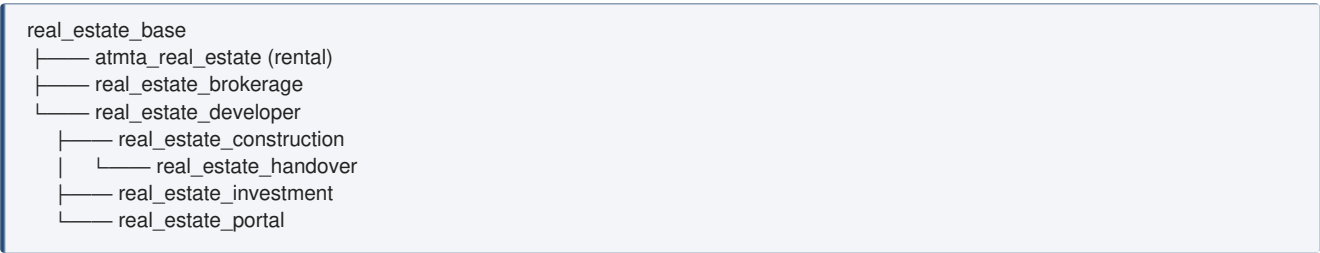
1. System Architecture
2. Master Flow: Developer Lifecycle (NEOM Pilot example)
3. Brokerage Flow: Sell a Property
4. Rental Flow: Lease a Unit
5. Customer Portal Flow
6. Investment Analysis Walkthrough
7. Module-by-Module Reference
8. Daily Automation (Crons)
9. Quick-Start Checklist

1. System Architecture

The system is a stack of 8 modules. Each row in the table below depends only on rows above it.

Tier	Module	Owns
Foundation	real_estate_base	Property catalog, hierarchy (Compound → Building → Floor → Unit → Room), maintenance, properties map dashboard, Leaflet stack, common security groups
Business Domains	atmta_real_estate (rental)	Lease contracts, payment plans, auto-invoicing, security deposit, rental history
	real_estate_brokerage	Listings, leads, viewings, offers, transactions, commissions
	real_estate_developer	Projects → Phases, reservations, installment plans, sale contracts
Operations	real_estate_construction	Milestones, contractors, cost vs budget, weighted progress
	real_estate_handover	Handover events, checklist, snagging, warranty
Analytics & Portal	real_estate_investment	NPV / IRR / DCF feasibility studies + scenarios
	real_estate_portal	Customer-facing pages: contracts, installments, snagging

Dependency Graph



The Spine: `realestate.property`

Every other module either adds fields to the property model or links to it. A property knows:

- Its **hierarchy** (parent / children) and level (Compound / Building / Floor / Unit / Room)
- Its **project & phase** (added by developer module)
- Its **reservations & sale contracts** (developer)
- Its **rental history & rental statistics** (rental)
- Its **active sales listing** (brokerage)
- Its **readiness percentages** (handover)
- Its **location, owner, manager, images, maintenance tickets** (base)

2. Master Flow — Developer Lifecycle

This is the longest end-to-end flow and exercises the most modules. Example scenario: **NEOM Pilot District** — a residential project in Tabuk with 80 units to sell off-plan.

Feasibility → Project & Phases → Units → Construction → Reservation → Sale Contract → Installments → Handover → Warranty → Portal

Step 1 — Run a Feasibility Study

1 Open [Investment → Feasibility Studies → New](#)

1. Set `project_id` = "NEOM Pilot District" (optional but recommended).
2. Set `discount_rate` = 10%, `analysis_period_years` = 7.
3. Click **Pull from Project** — the system auto-seeds Year 0 outflow = project budget and spreads the expected revenue across Years 1–7.
4. NPV, IRR and payback period compute live in the header. A green **POSITIVE** ribbon appears if NPV > 0.
5. On the **Scenarios** tab, add three rows:
 - "Best Case" — `inflow_factor` 1.15, `outflow_factor` 0.95
 - "Expected" — both 1.0 (baseline)
 - "Worst Case" — `inflow_factor` 0.85, `outflow_factor` 1.15

Each row gets its own scenario NPV and IRR.

6. If the worst case is still positive, click **Approve**.

Step 2 — Set Up the Project

2 Open [Developer → Projects → NEOM Pilot District](#)

1. Confirm `project_type` = "Mixed-use", `expected_budget`, `expected_revenue` and dates.
2. On the **Phases** tab, define Phase 1 and Phase 2 (each with expected delivery dates).
3. Click **Start Construction** — the project state moves from `planning` to `construction`.
4. Stat buttons show **Phases (2)** and **Units (0)**.

Step 3 — Create the Unit Inventory

3 Open [Real Estate → Properties → All Properties → New](#)

1. Create a Compound: `hierarchy_level` = compound, set `project_id` + `phase_id`, set latitude/longitude via the Map tab (click on the map).
2. Inside that Compound, create Building 1 — set `parent_id` to the Compound, `hierarchy_level` = building.
3. Inside each Building, create Units (`hierarchy_level` = unit, `parent_id` = building). Set `base_price` on each.
4. The Compound's **occupancy state** auto-rolls up from its leaf units.

Note: The seeded demo data already has Riyadh Heights and Jeddah Corniche populated with this exact pattern (2 compounds → 4 buildings → 20 units + 4 villas).

Step 4 — Track Construction

4 Open [Construction → Milestones → New](#)

1. For each major construction phase, create a milestone: name, category (foundation/structure/MEP/finishing/etc.), `weight` (% of total project), expected dates, contractor, budget.
2. Assign a Contractor (create via `Construction → Contractors` if needed).
3. As work progresses, update `completion_percentage`. The state auto-transitions:
 - 0% with future end date → `not_started`
 - 1–99% → `in_progress`
 - 100% → `completed`
 - Past end date with <100% → `delayed`
4. Log each cost as a `Cost Line`: amount, category (material/labor/equipment), vendor bill link. The project's `actual_cost` and `cost_variance` recompute live.
5. The project's `construction_progress` is a weighted average of all its milestones' completion percentages.

Step 5 — Open the Project for Sales

5 From the project form, click Open for Sales

State changes to `marketing`. Units with `state` = available become reservable.

Step 6 — Define an Installment Plan

6 Open `Developer → Configuration → Installment Plans → New`

Example plan for 5-year off-plan sale:

Field	Value
Plan Name	5-Year, 20% down + balloon
Down Payment Method	Percentage
Down Payment Value	20%
Recurrence	Quarterly
Number of Installments	16 (4 years)
Grace Period	3 months
Balloon Method	Percentage
Balloon Value	30%
Balloon at Handover	Yes

Step 7 — Reservation

7 Open `Developer → Reservations → New`

1. Pick a unit (only `state` = available works cleanly), buyer (res.partner), and agent.
2. Click **Save** — `hold_started_at` defaults to now; `hold_expiry_at` auto-computes from `hold_duration_hours` (default 72).
3. The unit moves to `reserved` automatically.
4. State chain: `hold` → **Confirm Booking** (after booking fee paid) → `booked`.
5. If the hold expires (cron runs hourly), it transitions to `expired` and the unit is freed back to `available`.

Step 8 — Sale Contract with Auto-Generated Schedule

8 From the booked reservation, click Create Sale Contract

1. A draft `realestate.sale.contract` opens, pre-filled with unit, buyer, agent, proposed price, plan.
2. Set `contract_date` and `expected_handover_date`.
3. Click **Generate Schedule** — the system invokes `installment_plan.compute_schedule()` :
 - Row 1: Down payment (sequence 1, kind=down, date = `contract_date`)
 - Rows 2–17: Quarterly installments (kind=installment, dates spaced by 3 months after the grace period)
 - Row 18: Balloon (kind=balloon, date = `expected_handover_date`)
4. Click **Sign Contract** — state moves to `signed`. Property state → rented (occupied semantic).

Step 9 — Automated Installment Invoicing

The daily cron `cron_auto_invoice_due_installments` runs every morning. It looks for installments where:

- State is `pending`
- Due date is within `real_estate_developer.installment_lead_days` (default 7 days)
- Parent sale contract state is signed or handed_over

For each match, it creates a draft `account.move` (out_invoice) on the buyer's partner, with one line for the unit's product variant and amount = installment amount. The installment transitions to `invoiced` and its `move_id` points at the invoice.

Once the buyer pays, click **Mark Paid** on the installment line. The contract's `paid_amount`, `balance_due` and `progress_bar` update.

Step 10 — Handover

10 When construction nears completion, open `Handover → Handovers → New`

1. Pick the sale contract — property, buyer, project auto-fill.
2. Pick the `template_id` = "Standard Residential Handover" (seeded).
3. Click **Load Template Items** — 9 checklist items appear (utilities/inspection/cleaning/documents/keys/safety).
4. Set `scheduled_date` and `warranty_period_months` (default 12).
5. On the day, click **Start Inspection**. Walk the checklist with the buyer, marking items **Done** / **Skipped** / **Failed**.
6. If you find defects, click **Mark Snagging** and create snagging issues on the Snagging tab. Each has:
 - `severity` : minor / major / critical
 - `location_on_unit`, photo, assigned contractor
 - State chain: open → assigned → in_progress → resolved → verified
7. Once all snagging is resolved & verified, click **Complete Handover**:
 - Handover state → `completed`
 - A warranty record is auto-created (`start_date` = today, `end_date` = today + N months)
 - The sale contract auto-transitions to `handed_over`
 - The property's `owner_id` changes from the developer's company to the buyer's partner

Step 11 — Warranty & Aftercare

A warranty record now lives at `Handover → Warranties`. State auto-computes as `active` until `end_date` passes, then `expired` (daily cron). If a buyer reports a defect during the warranty period via the portal, a new snagging issue is created linked back to the handover.

3. Brokerage Flow — Sell a Resale Property

This flow uses the `real_estate_brokerage` module, which is independent of the developer flow. It's typically used to broker resale of existing units between two private parties (or by a development company selling its own inventory through agents).

Lead → Match → Viewing → Offer → Transaction → Commission

Step 1 — Capture the Listing

1 Open [Brokerage → Listings → New](#)

- Pick `property_id` (must already exist in the catalog).
- Set `list_price`, `expiry_date`, `lister_agent_id`.
- Marketing description (Html) and channel tags (Bayut / Property Finder / etc.).
- Click **Activate** — state moves from `draft` to `active`; `list_date` stamps.

Step 2 — Capture a Lead

2 Open [Brokerage → Leads → New](#)

- Contact (res.partner), source (walk-in / website / referral / portal / call).
- Preferences: budget range, area range, bedrooms, property types, preferred city/district.
- Click **Qualify** — the lead's state moves to `qualified`.

Step 3 — Auto-Match Listings

3 On the lead, click Find Matches

The system searches active listings using the lead's preferences (budget, type, city), populates the `matched_listing_ids` tab, and moves the lead to state `matched` if any matches were found.

Step 4 — Schedule a Viewing

4 Click Schedule Viewing

- A new `realestate.viewing` opens pre-filled with listing, lead, buyer, agent.
- Set `scheduled_at` and `duration` (default 30 min).
- The viewing shows on the Calendar view ([Brokerage → Viewings → Calendar](#)) for the agent.

Step 5 — Complete the Viewing & Capture an Offer

5 After the showing, click Mark Completed, set feedback rating + next action, then Create Offer.

An offer form opens pre-filled. Capture:

- `amount`, `offer_date`, `expiry_date`
- `conditions` (subject to financing, inspection, etc.)
- `deposit_amount` (earnest money)
- State: `submitted`

Step 6 — Accept the Offer

6 Click **Accept** on the winning offer.

Cascading effects:

- Other open offers on the same listing → auto-rejected with reason "other_offer_accepted"
- Listing state → **under_offer**
- Lead state → **offer**

Step 7 — Create the Transaction

7 On the accepted offer, click **Create Transaction**

A new `realestate.transaction` opens. Two flavors:

transaction_type	Behavior
brokerage	Commission lines required before close (you're brokering for another owner)
in_house	Commission lines optional (you're selling your own property)

Workflow:

1. Set `deposit_amount` → click **Record Deposit** → state **deposit_received**.
2. Click **Sign Contract** → state **contract_signed**; `sale_agreement_date` stamps.
3. Click **Add Default Commissions** — auto-creates commission lines for the lister and selling agents using their `commission_share_default`.
4. Click **Close**:
 - Listing → **sold**
 - Property's `owner_id` → buyer
 - Transaction state → **closed**; `closing_date` stamps

Step 8 — Track Commission Payouts

On the Commission tab of a closed transaction, each agent's commission line shows amount + paid flag. Once you pay an agent (e.g., via a vendor bill), tick `paid` and link the `payment_move_id`.

4. Rental Flow — Lease a Unit

This flow uses `atmta_real_estate` (rental) and is independent of brokerage. Typical use: a landlord or property manager leasing units.

Property → Contract → Payment Schedule → Deposit → Auto-Invoicing → Maintenance

Step 1 — Property & Tenant Setup

Property already in the catalog (from base module). Tenant is a `res.partner` — create or pick an existing one.

Step 2 — Define a Payment Plan

2 Open `Real Estate → Configuration → Payment Plans → New`

Payment plans here are simpler than developer installment plans — they define a recurrence (monthly / quarterly / etc.) and an interval after lease start.

Step 3 — Create the Rental Contract

3 Open `Real Estate → Contracts → All Contracts → New`

- Tenant (partner), start/end dates, base rent.
- For multi-unit contracts: add lines (one per property unit) with their own payment plans and increment/discount rules.
- Optional: define increment rules on the contract (e.g., 5% rent increase starting in month 13, lasting 12 months).
- Click **Confirm** → state `confirmed` → payment schedule auto-generates on `realestate.contract.payment` rows.
- Each scheduled payment row carries: `date_due`, `hijri_date_due`, `amount`, `state=draft`.

Step 4 — Record Security Deposit

4 On the contract form, open the Security Deposit tab

1. Set `deposit_amount`.
2. Click **Record Deposit** → state moves to `held`; `deposit_paid_date` stamps.
3. At lease end:
 - **Refund (Full)** → state `refunded`
 - **Refund (Partial)** → set `refund_amount` first, then click. State `partial_refund`. The forfeit amount auto-computes as the gap.
 - **Forfeit** → state `forfeited`

Step 5 — Auto-Invoicing

The daily cron `cron_auto_invoice_due_payments` drafts invoices for upcoming payments. Configurable lead window (default 7 days). Each created `account.move` is linked back to the payment row via `move_id`; the payment moves to `invoiced`.

Step 6 — Maintenance Tickets

If the tenant reports an issue, log a maintenance request:

6 Open `Real Estate → Maintenance → Requests → New`

Lifecycle: **draft** → **scheduled** → **in_progress** → **done**. When **in_progress**, the property's state can be set to **maintenance** ; on done, it returns to **available** (if no other active maintenance).

5. Customer Portal Flow

Buyers can self-serve via the portal. They must have a portal user account whose `partner_id` matches a buyer on a sale contract.

Available Pages

URL	What the customer sees
<code>/my</code>	Portal home with 3 cards: Sale Contracts, Installments, Snagging Issues, each with a count
<code>/my/sale-contracts</code>	List of all sale contracts where they are the buyer, with status badge and balance
<code>/my/sale-contracts/<id></code>	Detail view: unit, project, dates, sale price, paid, balance, progress bar, full installment table
<code>/my/installments</code>	All installments across all their contracts with status badges; totals at the top
<code>/my/snagging</code>	Their reported defects with severity and resolution status
<code>/my/snagging/new</code>	Form to report a new defect: pick handover, location on unit, severity, description → creates a <code>realestate.snagging.issue</code> linked to the handover

Typical Customer Journey

1. **Logs in** at `localhost:8069` , lands on `/my` .
2. **Clicks "Sale Contracts"** → sees their unit, sale price 850,000 SAR, paid 170,000 (20% down), balance 680,000.
3. **Drills into the contract** → sees the full installment schedule: down (paid), 16 quarterly installments (next due 2026-08-15, 42,500 SAR), 1 balloon (255,000 SAR at handover).
4. **Logs a snagging issue** a week after handover: "Hallway light fixture loose" → pick handover, location "Master corridor", severity minor → submits.
5. Developer's snagging dashboard shows the new issue. Operations assigns a contractor → resolves → verifies. Customer sees the state transitions on their snagging page.

6. Investment Analysis Walkthrough

The `real_estate_investment` module computes NPV, IRR, and payback for project-level cash flows. Two ways to feed it:

Method A — Pull from Project Estimates

If the project has `expected_budget` and `expected_revenue` set, click **Pull from Project**:

- Year 0: outflow = `expected_budget`, inflow = 0
- Years 1..N: inflow = `expected_revenue` / N (evenly split), outflow = 0

Tweak rows manually after for more realism (e.g., front-load revenue in years 2–3, add operating costs as outflows in years 4–7).

Method B — Manual Entry

Click **Seed Empty Years** to pre-populate N+1 zero-value rows. Type values year by year.

The Math

For a set of net cash flows $CF_0, CF_1, \dots, CF_N$ and discount rate r :

$$\text{NPV} = \sum CF_t / (1+r)^t \text{ for } t = 0 \dots N$$
$$\text{IRR} = \text{the rate } r^* \text{ such that } \text{NPV}(r^*) = 0$$

IRR is computed via bisection (no external numpy dependency). Payback period is the year in which cumulative cash flow turns positive, with linear interpolation within the year.

Scenario Analysis

Each scenario applies multipliers to all inflows and outflows of the base study, plus an optional discount rate override. So a "Cost Overrun" scenario with `outflow_factor` = 1.20 simulates 20% higher building costs. The scenario's NPV and IRR are computed independently.

Typical practice: create three scenarios (Best / Expected / Worst) for any feasibility. If all three are NPV-positive, the project is robust. If only the Best case is positive, it's fragile.

7. Module-by-Module Reference

real_estate_base

Model	Purpose
realestate.property	Universal property record (delegated to product.template). 5-level hierarchy, occupancy state roll-up, lat/lng.
property.type / property.usage	Classification lookups.
property.image	Photo gallery.
realestate.maintenance.request	Ticket: draft → scheduled → in_progress → done.

atmta_real_estate (rental)

Model	Purpose
realestate.contract	Rental contract with payment schedule, increment rules, security deposit.
realestate.contract.line	Per-unit line for multi-unit contracts.
realestate.payment.plan	Recurrence template.
realestate.contract.payment	Auto-generated payment-due rows.
realestate.contract.increment.rule	Stepped rent escalation.
realestate.contract.utility.line	Utility bill against a contract.
realestate.property.rental.history	Audit trail.

real_estate_brokerage

Model	Purpose
realestate.listing	Property for sale (draft → active → under_offer → sold).
realestate.lead	Buyer prospect with preferences.
realestate.viewing	Scheduled showing.
realestate.offer	Bid (submitted → accepted/rejected).
realestate.transaction	Closed sale (in-house OR brokerage).
realestate.commission	Per-line agent commission.

real_estate_developer

Model	Purpose
realestate.project	Top-level development.
realestate.phase	Sub-phase with own state machine.
realestate.installment.plan	Reusable template (down + recurring + balloon).
realestate.unit.reservation	Hold-and-book with timer.
realestate.sale.contract	Off-plan sale lifecycle.
realestate.sale.installment	Per-line schedule entry.

real_estate_construction

Model	Purpose
realestate.contractor	Vendor with specialization.
realestate.construction.milestone	Project/phase milestone with weight + completion %.
realestate.construction.cost.line	Actual cost entry.

real_estate_handover

Model	Purpose
realestate.handover.checklist.template	Reusable checklist.
realestate.handover	Per-unit handover event.
realestate.handover.checklist.item	Live checklist row.
realestate.snagging.issue	Defect with severity + contractor.
realestate.warranty	Auto-created on handover completion.

real_estate_investment

Model	Purpose
realestate.investment.feasibility	Study with NPV / IRR / payback.
realestate.investment.cash.flow	Per-year cash flow row.
realestate.investment.scenario	Sensitivity factors.

real_estate_portal

No new models — controllers and QWeb templates only. Extends `portal.CustomerPortal` with 5 routes.

8. Daily Automation (Crons)

Cron	Frequency	What it does
Rental: Auto-invoice payments due soon	Daily	Drafts an invoice for each unbilled rental payment whose due date is within the lead window (default 7 days).
Real Estate: Expire Listings	Daily	Moves listings past their expiry_date from active to expired.
Real Estate: Expire Offers	Daily	Moves offers past their expiry_date to expired.
Developer: Expire Unit Holds	Hourly	Reservations on hold past hold_expiry_at → expired; frees the unit back to available.
Developer: Auto-invoice installments due soon	Daily	Drafts invoices for sale installments within lead window.
Handover: Update warranty states	Daily	Marks warranties whose end_date has passed as expired.
Contract Line Auto Expiry (rental)	Daily	Auto-expires contract lines past their end_date.

9. Quick-Start Checklist

A practical checklist to onboard a new tenant of this system.

#	Task	Menu
1	Create your developer/management company partner	Contacts
2	Create users + assign Real Estate + Developer manager groups	Settings → Users
3	Define property types (Apartment / Villa / Office / etc.)	Real Estate → Properties → Property Types
4	Create your first project (state = planning)	Developer → Projects
5	Add phases under the project	Developer → Phases
6	Run a feasibility study (NPV / IRR check)	Investment → Feasibility Studies
7	Approve the study → move project to construction	Developer → Projects
8	Set up construction milestones with weights	Construction → Milestones
9	Register contractors	Construction → Contractors
10	Create the property hierarchy (Compound → Building → Unit)	Real Estate → Properties
11	Define installment plans	Developer → Configuration → Installment Plans
12	Open project for sales (state = marketing)	Developer → Projects
13	Train agents to use Reservations → Sale Contracts flow	Developer
14	Verify daily crons are active in Scheduled Actions	Settings → Technical → Scheduled Actions
15	Set up portal users for buyers (one per partner)	Settings → Users & Companies → Portal Access
16	For rentals: add increment rules + payment plans	Real Estate → Configuration
17	Create handover checklist templates suited to your standard	Handover → Configuration → Checklist Templates

Tip: Use the seeded demo data (Riyadh Heights and Jeddah Corniche projects, 30 properties, 30 partners, 22 projects total) to walk through every flow end-to-end before importing real data. Drop the database and re-seed via the seed scripts in /tmp to start over.

Common Operational Questions

Question	Answer
What if a buyer wants to cancel after signing?	Open the sale contract → Cancel button. Frees the unit; existing installments stay for record but the contract's state is cancelled.
How do we handle a buyer transferring their contract to another buyer?	Not in MVP. Workaround: cancel the original contract and create a new one for the new buyer.
What if construction is delayed past handover date?	The milestone auto-transitions to "delayed" state. Update expected_handover_date on the sale contract and the balloon installment moves accordingly.
Can a single property be both for-sale and for-rent?	Yes — a property can have an active brokerage listing AND a rental contract simultaneously. The state field reflects what's most "live" (rented / reserved / available).
How are commissions paid to agents in practice?	On the Commission line, tick Paid and link a vendor bill (account.move with move_type=in_invoice) on the agent's partner record.

— End of Working Flows Guide —